

香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

# Operating Systems
# Lecture 7

# Concurrency and its problems

**Prof. Yunming XIAO**

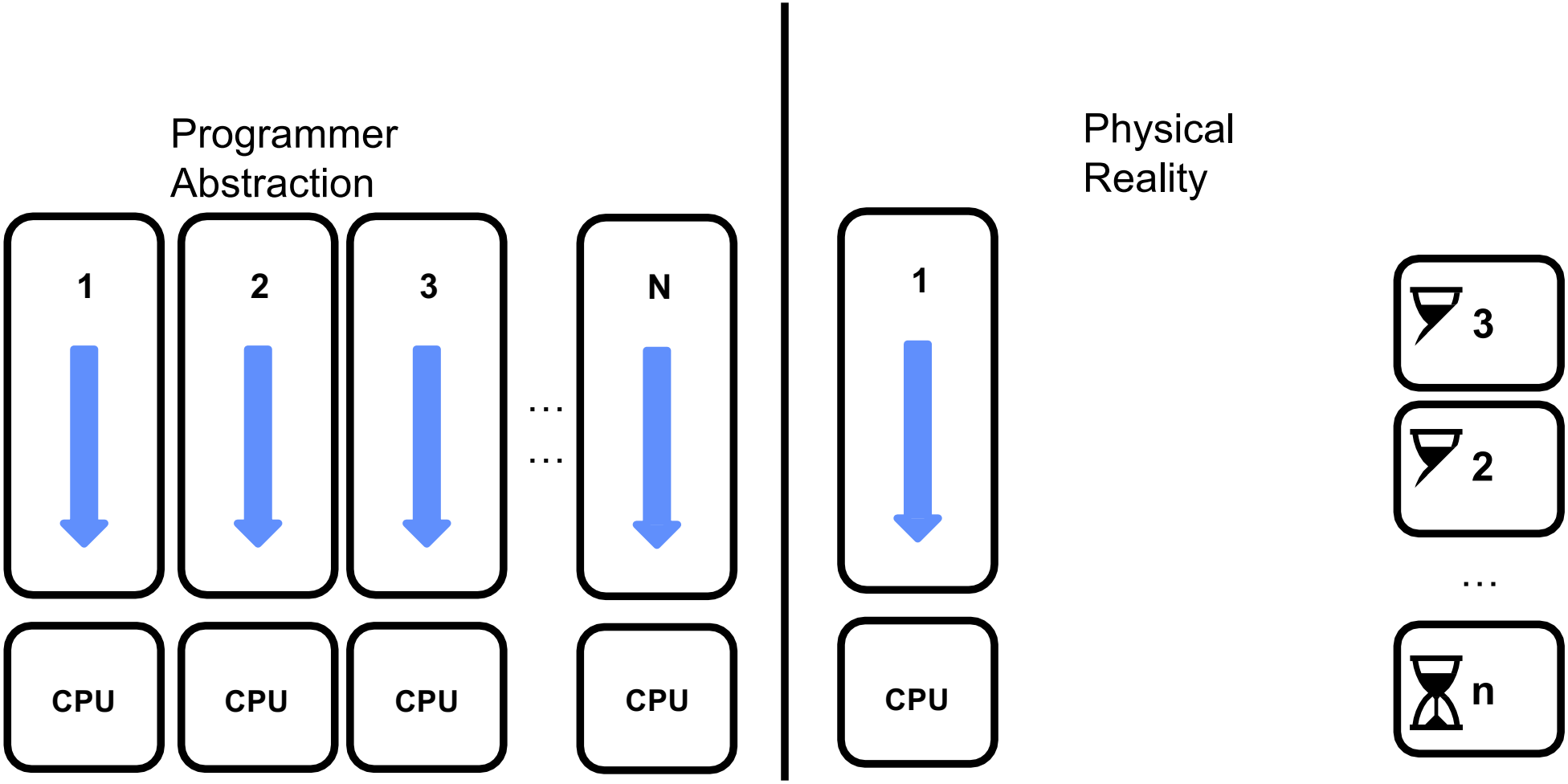
**School of Data Science**

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

# Goals for today

- Threads recap
- Challenges and pitfalls of concurrency
- Synchronization operations / critical sections
- How to build a lock?
- Atomic instructions

# What is a thread?



# Recall: thread ≠ process

- Processes defines the granularity at which the OS offers isolation and protection
- Threads capture concurrent sequences of computation
- Processes consist of one or more threads!

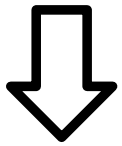
Process
Protection

Thread
Concurrency

# Implementing threads

**No protection**

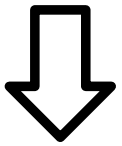
Threads inside the same process and are not isolated from each other



Share an address space & share IO state (FDs)

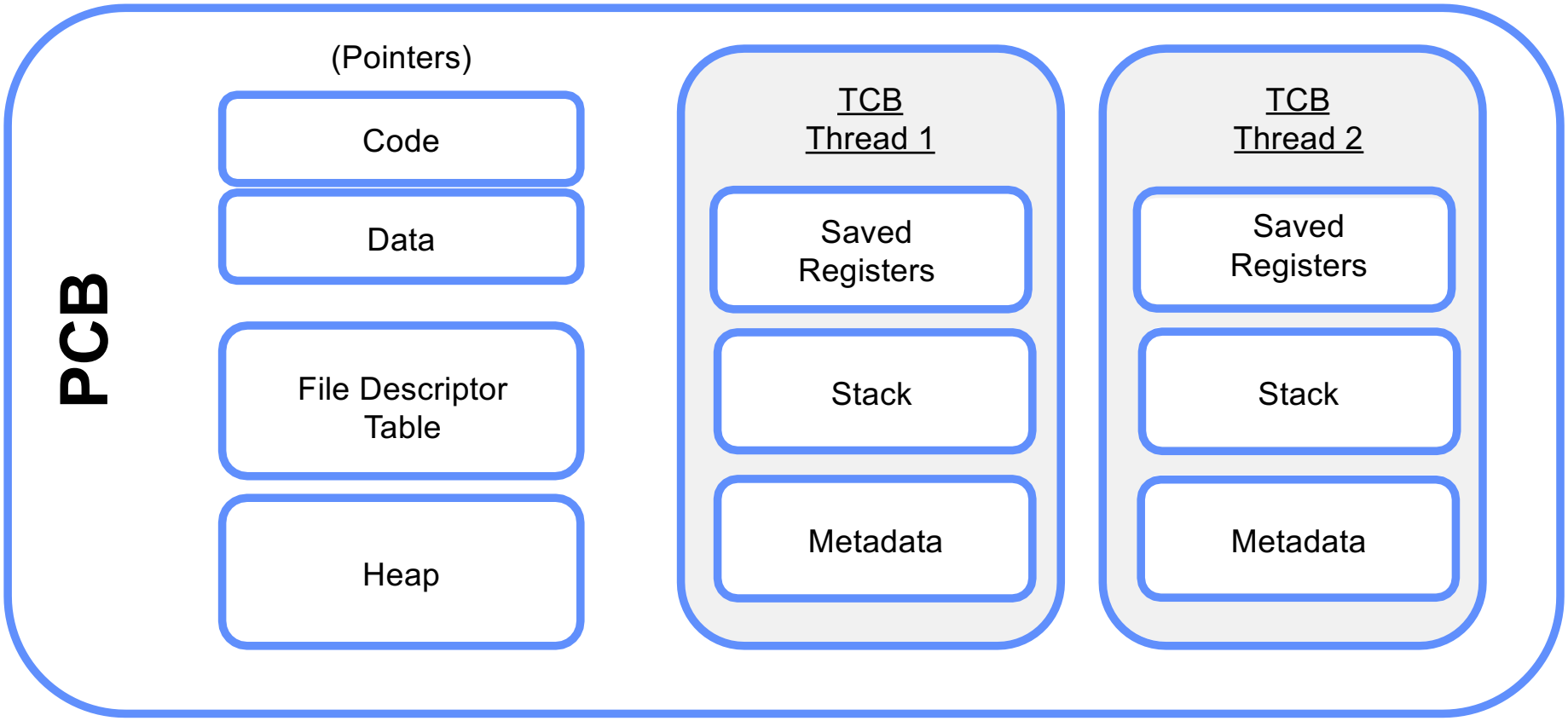
**Individual execution**

Threads execute disjoint instruction streams. Need own execution context



Individual stack, register state

# Implementing threads



# Threads in Linux

Everything is a thread (`task_struct`)

Scheduler only schedules `task_struct`

**To fork a process:**

**Invoke** `clone(…)` **syscall**

**To create a thread:**

**Invoke** `clone(CLONE_VM | CLONE_FS| CLONE_FILES | CLONE_SIGHAND, 0)`

CLONE_VM: Share address space
CLONE_FS: share file system
CLONE_FILES: share open files
CLONE_SIGHAND: share handlers with parents

**Processes are better viewed as the containers in which threads execute**

# OS library API for threads (pthreads)

```
int pthread_create(pthread_t *thread, …
                   void *(*start_routine)(void*), void *arg);
```

- Creates a thread that runs `start_routine`

```
void pthread_exit(void *value_ptr);
```

- Terminates calling thread and returns value_ptr to any successful join call

```
int pthread_join(pthread_t thread, void **value_ptr);
```

- Suspends execution of calling thread until target thread terminates

```
int pthread_yield();
```

- Makes calling thread yield the CPU to other threads

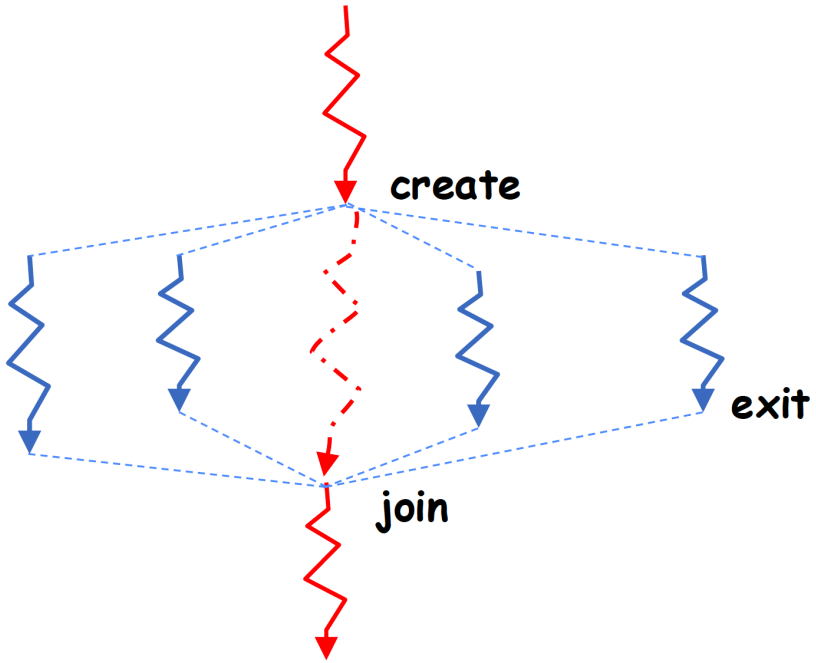
# pthreads example

```
void *mythread(void *arg) {
  printf("%s\n", (char *) arg);
  return NULL;
}

int main(int argc, char *argv[]) {
  pthread_t p1, p2;
  printf("main: begin\n");
  pthread_create(&p1, NULL, mythread, "A");
  pthread_create(&p2, NULL, mythread, "B");
  // join waits for the threads to finish
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("main: end\n");
}
```

What does this print?

# Fork-join pattern




- Main thread *creates* (forks) collection of sub-threads passing them args to work on…
- … and then *joins* with them, collecting results.

# Example: multithreaded web server

```
// Socket setup code elided…

while (1) {
  // Accept a new client connection, obtaining a new socket
  pthread_t tid;
   int conn_socket = accept(server_socket, NULL, NULL);
   int* arg = (int*) malloc(sizeof(int));
   *arg = conn_socket;
   pthread_create(&tid, NULL &serve_client, &arg);
}
close(server_socket);
```

# Inside the pthread_create(…) call

| | |
|---|---|
| Do some work like a normal fn…<br>Place syscall # into %eax<br>Put args into registers %ebx, …<br>Special trap instruction | OS Library |
| Mode switches & switches to kernel stack.<br>Saves recovery state<br>Jump to interrupt vector table at location 128.<br>Hands control to syscall_handler | CPU |
| Use %eax register to index into system call dispatch table. Invoke do_fork() method. Initialize new TCB. Mark thread READY. Push errcode into %eax | Kernel |
| Restore recovery state and mode switch | CPU |
| Get return values from regs<br>Do some more work like a normal fn… | OS Library |

# With great power comes great concurrency

```
pthread_t tid[2];
int counter = 0;

void* do_something(void *arg) {
  unsigned long i = 0;
  for (int i = 0; i < 1000; i++) {
    counter += 1;
  }
  return NULL;
}

int main(void) {
  pthread_create(&(tid[0]), NULL, &do_something, NU
  pthread_create(&(tid[1]), NULL, &do_something, NU
  pthread_join(&(tid[0]), NULL);
  pthread_join(&(tid[1]), NULL);
  printf("Counter %d\n", counter);
  return 0;
}
```

What will be the final answer?

```
yunming@laptop> gcc concurrency.c -o
concurrency –pthread
```

```
yunming @laptop> ./concurrency
```

```
Counter 2000
```

```
yunming@laptop> ./concurrency
```

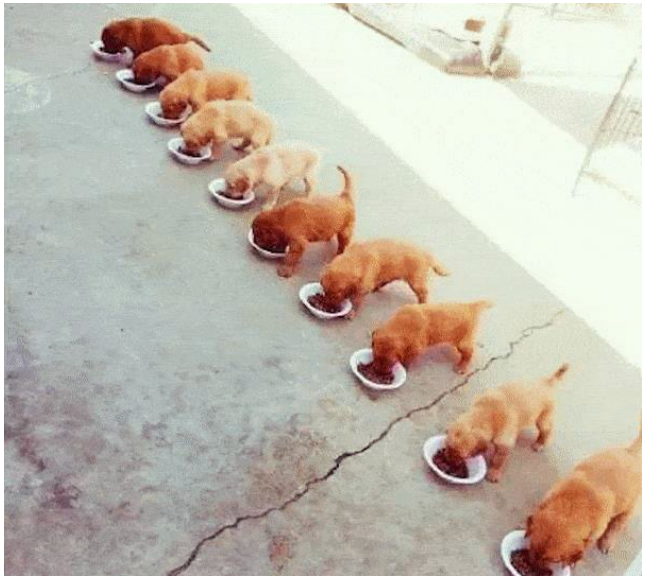
```
Counter 1937
```

```
yunming@laptop> ./concurrency
```

```
Counter 1899
```

# Multi-threaded programming in theory and in reality

Multi-threaded programming in theory:



Multi-threaded programming in practice:

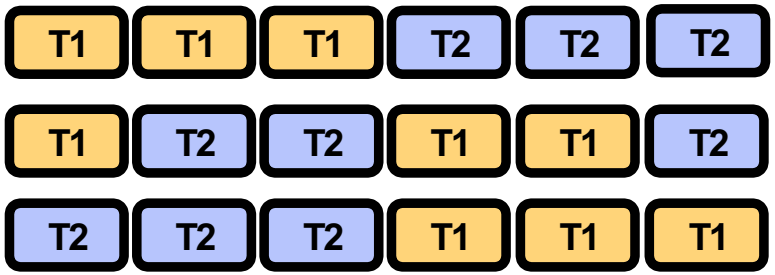


# With great power comes great concurrency

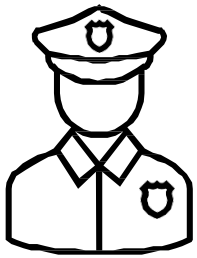
Protection is at the process level.

Threads not isolated.
Share an address space.

Non-deterministic interleaving of threads:



# With great power comes great concurrency



Public Enemy #1:

**THE RACE CONDITION**



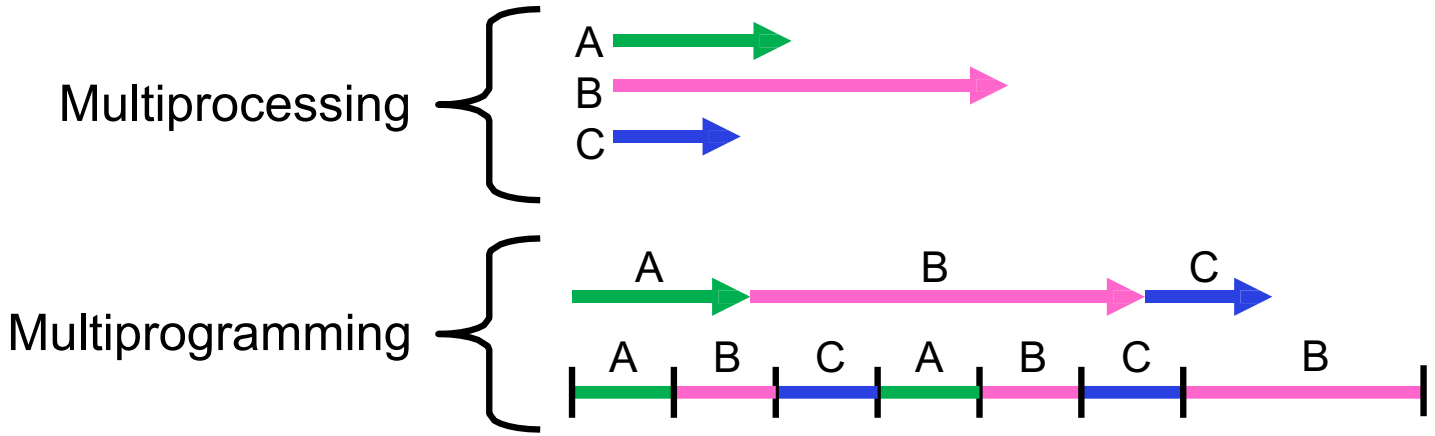
**Today and next three lectures: how can we regulate access to shared data across threads?**

# With great power comes great concurrency

- Multiprocessing → multiple CPUs
- Multiprogramming → multiple jobs or processes
- Multithreading → multiple threads per process

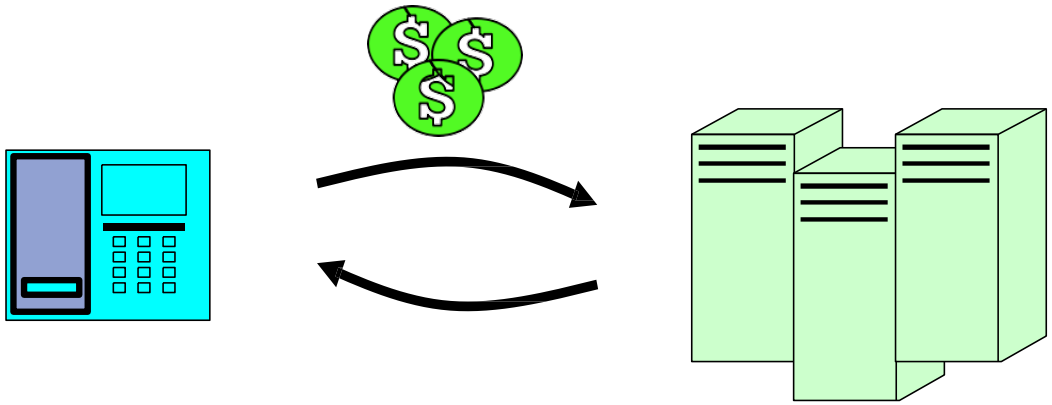
# Multiprocessing vs. multiprogramming

- What does it mean to run two threads “concurrently”?
  - Scheduler is free to run threads in any order
  - Can choose to run each thread to completion or time-slice in big or small chunks



# Example: ATM bank server

- Service a set a requests
- Do so without corrupting database
- Don’t hand out too much money



# Example: ATM bank server

- Suppose we wanted to implemented a server process to handle requests from an ATM network:

```
BankServer() {
  while (TRUE) {
    receive_request(&op, &acct_id, &amount);
    process_request(op, acct_id, amount);
}}

process_request(op, acct_id, amount) {
  if (op == deposit) deposit(acct_id, amount);
  else if …
}

deposit(acctId, amount) {
  acct = get_account(acct_id); /* may use disk I/O */
  acct->balance += amount;
  store_account(acct); /* requires disk I/O */
}
```

# Event driven version of ATM server

- Suppose we only had one CPU but still want to overlap I/O with computation. Without threads, we would have to rewrite in event-driven style:

```
BankServer() {
  while(TRUE) {
  event = wait_for_next_event();
  if (event == ATMRequest)
    start_on_request();
  else if (event == AcctAvail)
    continue_request();
  else if (event == AcctStored)
    finish_request();
  }
}
```

## Can threads make this easier?

- Threads yield overlapped I/O and computation without "deconstructing" code into non-blocking fragments

- One thread per request

- Requests proceed to completion, blocking as required

# Can threads make this easier?

- Suppose we wanted to implemented a server process to handle requests from an ATM network:

```
BankServer() {
  while (TRUE) {
    receive_request(&op, &acct_id, &amount);
    START_THREAD(process_request(op, acct_id, amount));
}}

process_request(op, acct_id, amount) {
  if (op == deposit) deposit(acct_id, amount);
  else if …
}

deposit(acctId, amount) {
  acct = get_account(acct_id); /* may use disk I/O */
  acct->balance += amount;
  store_account(acct); /* requires disk I/O */
}
```

# Remember the race condition…

- Shared state can go corrupted

Thread 1

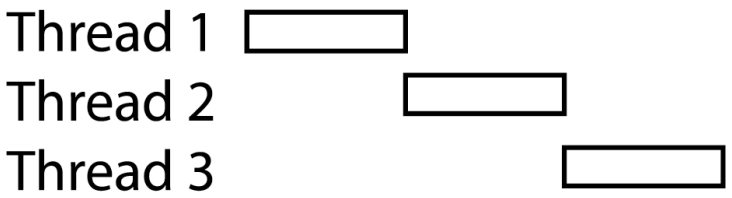
```
load r1, acct->balance


add r1, amount1
store r1, acct->balance
```

Thread 2

```
load r1, acct->balance
add r1, amount2
store r1, acct->balance
```

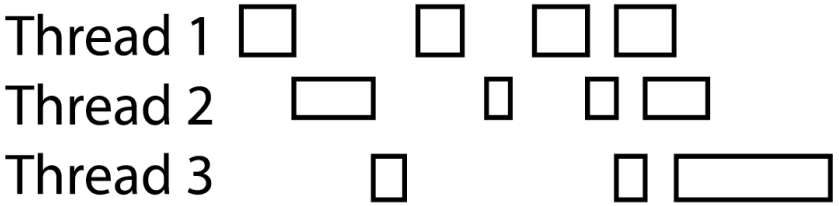
# Many possible executions




a) One execution




b) Another execution




c) Another execution

# Problem is at the lowest level

- Most of the time, threads are working on separate data, so scheduling doesn't matter

Thread 1

```
x = 1;
```

Thread 2

```
y = 2;
```

- However, what about (initially, y = 12):

Thread 1

```
x = 1;
x = y + 1;
```

Thread 2

```
y = 2;
y = y * 2;
```

- What if two threads are both writing to x?

# Atomic operations

- An operation that always runs to completion, or not at all
- It is indivisible: it cannot be stopped in the middle, and state cannot be modified by someone else in the middle
- Fundamental building block
- If no atomic operations, then have no way for threads to work together

# Atomic operations

- On most machines, memory references and assignments (i.e. loads and stores) of words are atomic

- Consequently – weird examples with partial writes to ints can't happen

- Many instructions are not atomic
  - Double-precision floating point store often not atomic
  - VAX and IBM 360 had an instruction to copy a whole array

# Another concurrent program example

- Two threads, A and B, compete with each other

Thread 1

```
i = 0;
while (i < 10)
  i = i + 1;
printf("A wins!");
```

Thread 2

```
i = 0;
while (i > -10)
  i = i - 1;
printf("B wins!");
```

- Assume that memory loads and stores are atomic, but incrementing and decrementing are not atomic
- What happens?

# Another concurrent program example

- Synchronization: Using atomic operations to ensure cooperation between threads

- Mutual Exclusion: Ensuring that only one thread does a particular thing at a time

- Critical Section: Piece of code that only one thread can execute at once. Only one thread at a time will get into this section of code

# API for locks

- Locks need to be allocated and initialized:
  - `pthread_mutex_t mylock;`
  - `mylock = PTHREAD_MUTEX_INITIALIZER;` or `pthread_mutex_init(&mylock, NULL);`

- Locks provide two **atomic** operations:
  - `acquire(&mylock)`: wait until lock is free; then mark it as busy
  - `release(&mylock)`: mark lock as free
    - Should only be called by a thread that currently holds the lock

# How would you fix the ATM problem?

- (No, getting rid of money is not an option for this class)

# Fix banking problem with locks!

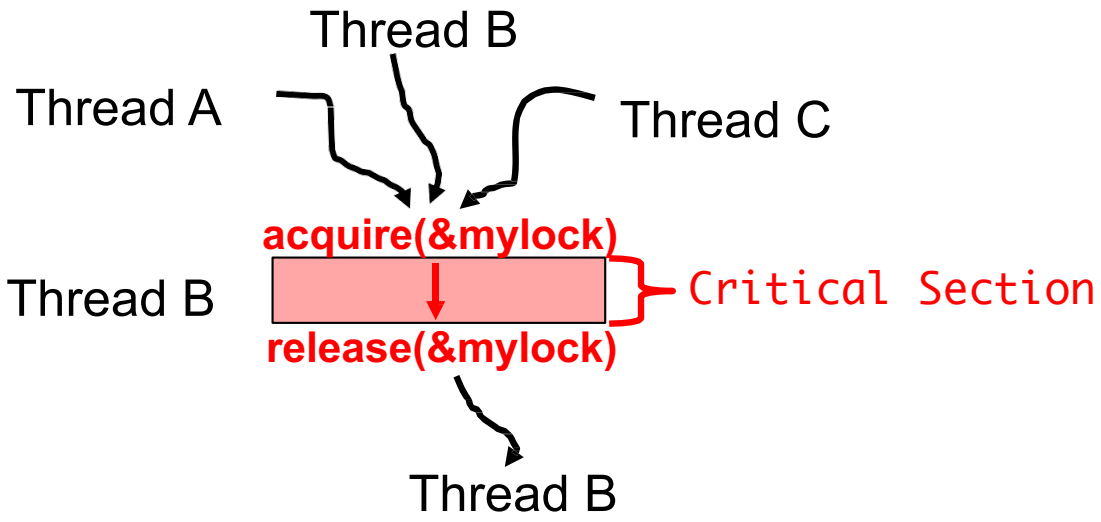
- Identify critical sections (atomic instruction sequences) and add locking

```
deposit(acctId, amount) {
  acquire(&mylock);        // What if someone else in critical section!
  acct = get_account(acct_id);
  acct->balance += amount;
  store_account(acct);
  release(&mylock);        // Release someone into critical section
}
```

Critical Section

# Fix banking problem with locks!

- Threads serialized by lock through critical section
- Only one thread at a time



# Correctness requirements

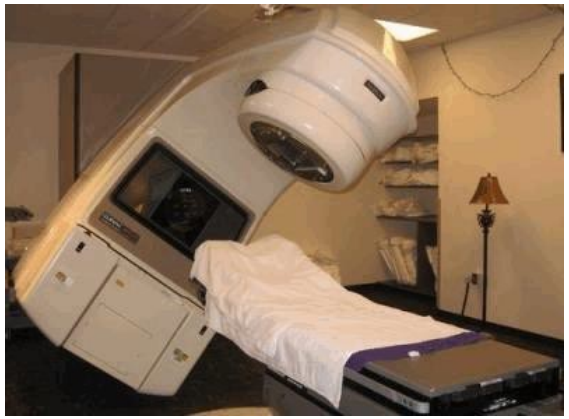
- Threaded programs must work for all interleavings of thread instruction sequences

- Cooperating threads inherently non-deterministic and non-reproducible

- Really hard to debug unless carefully designed!

# Therac-25

- Machine for radiation therapy
- Software control of electron accelerator and electron beam/Xray production
- Software control of dosage
- Concurrency bugs caused the death of several patients





# The importance of milk



# The important of milk

- Great thing about OS's – analogy between problems in OS and in real life
- Help you understand real life problems better
- But, computers are much stupider than people

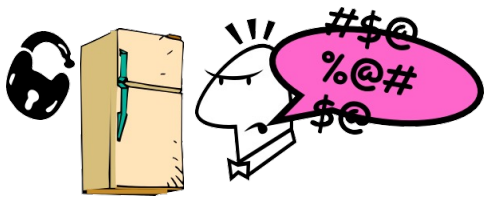
# Motivating example: “too much milk”

| Time | Person A | Person B |
|---|---|---|
| 3:00 | Look in Fridge. Out of milk | |
| 3:05 | Leave for store | |
| 3:10 | Arrive at store | Look in Fridge. Out of milk |
| 3:15 | Buy milk | Leave for store |
| 3:20 | Arrive home, put milk away | Arrive at store |
| 3:25 | | Buy milk |
| 3:30 | | Arrive home, put milk away |

# Solve with a lock?

- Lock prevents someone from doing something
  - Lock before entering critical section
  - Unlock when leaving
  - Wait if locked

- Fix the milk problem by putting a key on the refrigerator
- Lock it and take key if you are going to go buy milk
- Fixes too much: roommate angry if only wants *orange juice*



# Too much milk: correctness properties

- Need to be careful about correctness of concurrent programs, since they are non-deterministic
  - Impulse is to start coding first

# Too much milk: correctness properties

- Need to be careful about correctness of concurrent programs, since they are non-deterministic
  - Impulse is to start ~~coding~~ asking ChatGPT/Deepseek/Gimini first, then when it doesn't work, pull hair out
  - Instead, think first, then code!
  - Always write down desired behavior first

# Too much milk: correctness properties

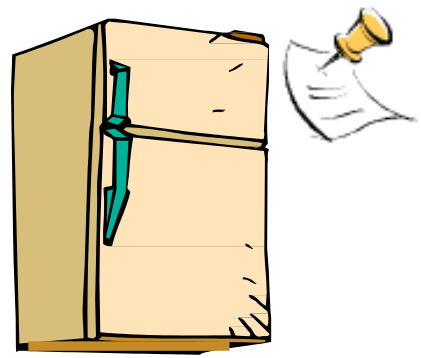
- What are the correctness properties for the “Too much milk” problem???
  - Never more than one person buys
  - Someone buys if needed

- First attempt: restrict ourselves to use only atomic load and store operations as building blocks

# Too much milk: solution #1

- Use a note to avoid buying too much milk:
  - Leave a note before buying (kind of “lock”)
  - Remove note after buying (kind of “unlock”)
  - Don’t buy if note (wait)
- Suppose a computer tries this (remember, only memory read/write are atomic)

```
if (no_milk) {
  if (no_note) {
    leave Note;
    buy milk;
    remove note;
  }
}
```



# Too much milk: solution #1

Thread 1

```
if (no_milk) {


  if (no_note) {
    leave note;
    buy milk;
    remove note;
  }
}
```

Thread 2

```
if (no_milk) {
  if (no_note) {

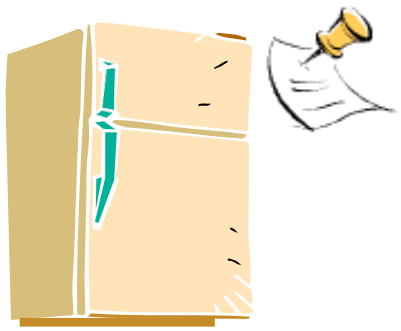

    leave note;
    buy milk;
    remove note;
  }
}
```

# Too much milk: solution #1

- Still too much milk but only occasionally!

- Thread can get context switched after checking milk and note but before buying milk!

- Solution makes problem worse since fails intermittently
  - Makes it really hard to debug…
  - Must work despite what the scheduler does!



# Too much milk: solution #1½

- Let's try to fix this by placing note first

```
leave note;
if (no_milk) {
  if (no_note) {
    buy milk;
  }
}
remove note;
```

- What happens here?
  - Well, with human, probably nothing bad
  - With computer: no one ever buys milk

# Too much milk: solution #2

- How about labeled notes?
  - Now we can leave note before checking

- Algorithm looks like this:

Thread 1

```
leave note_A;
if (no_note_B) {
  if (no_milk) {
    buy milk;
  }
}
remove note_A;
```

Thread 2

```
leave note_B;
if (no_note_A) {
  if (no_milk) {
    buy milk;
  }
}
remote note_B;
```

# Too much milk: solution #2

- Possible for neither thread to buy milk
  - Context switches at exactly the wrong times can lead each to think that the other is going to buy

- Really insidious:
  - Extremely unlikely this would happen, but will at worst possible time
  - Probably something like this in UNIX

# Too much milk: solution #2 problem

*I'm* not getting milk, *you're* getting milk

This kind of lockup is called "starvation!"

# Too much milk: solution #3

Thread 1

```
leave note_A;
while (note_B) { //X
  do nothing;
}
if (no_milk) {
  buy milk;
}
remove note_A;
```

Thread 2

```
leave note_B;
if (no_note_A) { //Y
  if (no_milk) {
    buy milk;
  }
}
remove note_B;
```

# Too much milk: solution #3

- Both can guarantee that:
  - It is safe to buy, or
  - Other will buy, ok to quit

- At X:
  - If no note B, safe for A to buy,
  - Otherwise wait to find out what will happen

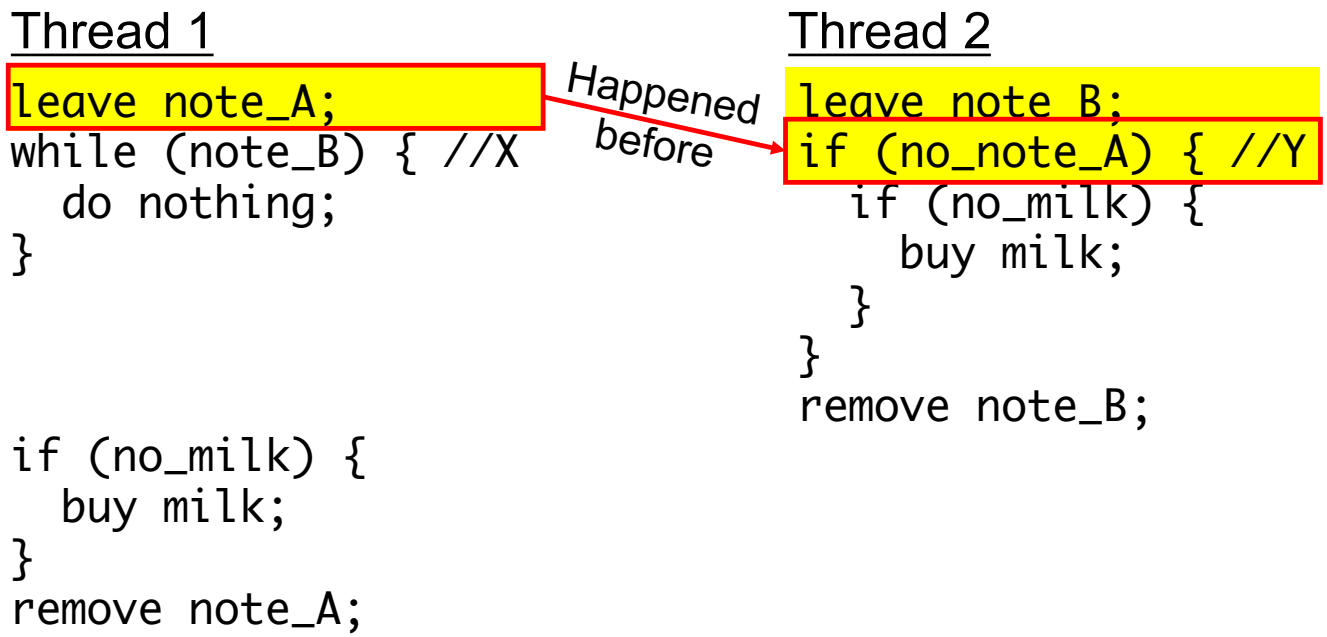
- At Y:
  - If no note A, safe for B to buy
  - Otherwise, A is either buying or waiting for B to quit

## To reason about whether this is correct

- Need some details about how our computer's memory works (a "memory model")
- We already assumed that individual loads and stores are atomic
- What can we assume about a series of loads and stores by our threads?
- In this example, we'll assume **sequential consistency**, which means:
  - There is a single, global order of loads and stores, and the operations issued by each thread are processed in program order
- For example, when Thread A does "buy milk" followed by "remove note A", Thread B will be able to "see" the milk before it "sees" that note A is gone

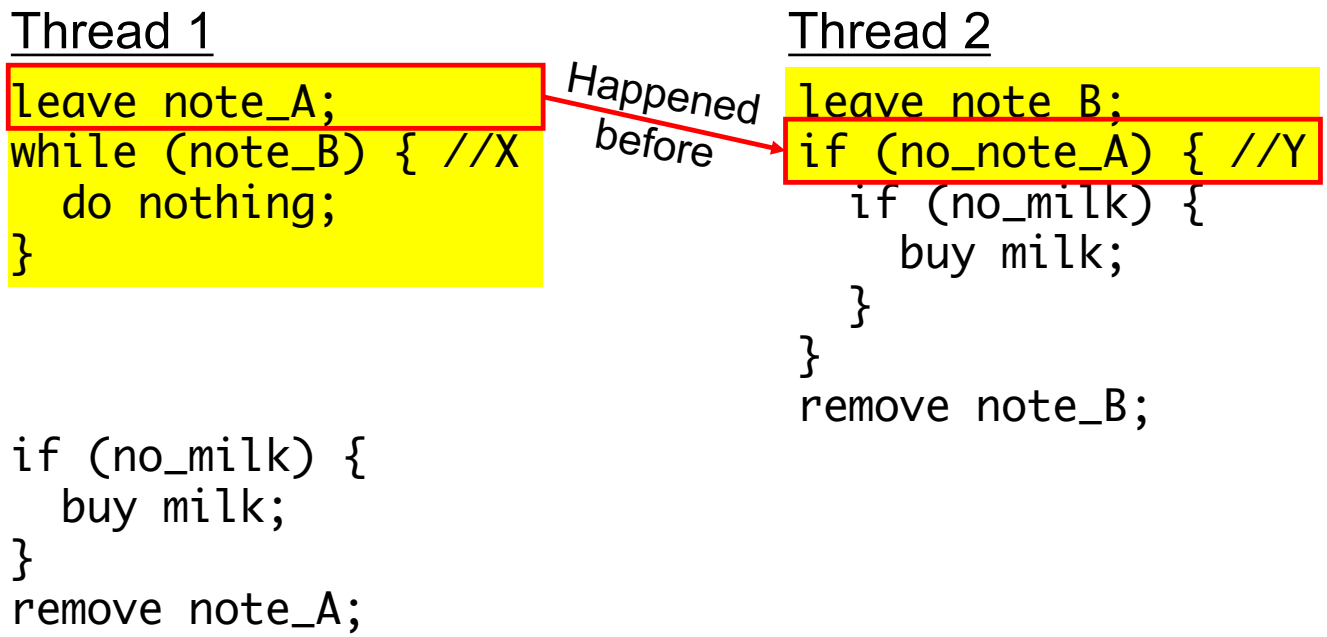
# Case 1

- "`leave note_A`" happens before "`if (no_note_A)`"



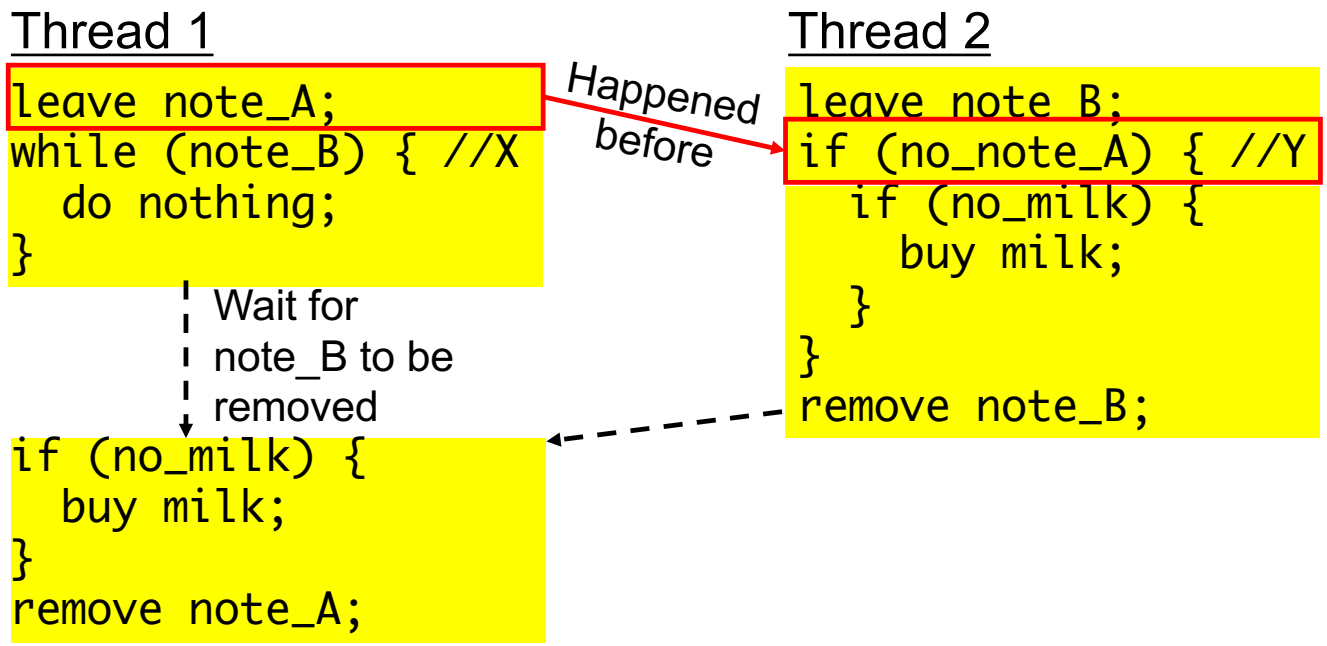
# Case 1

- "`leave note_A`" happens before "`if (no_note_A)`"



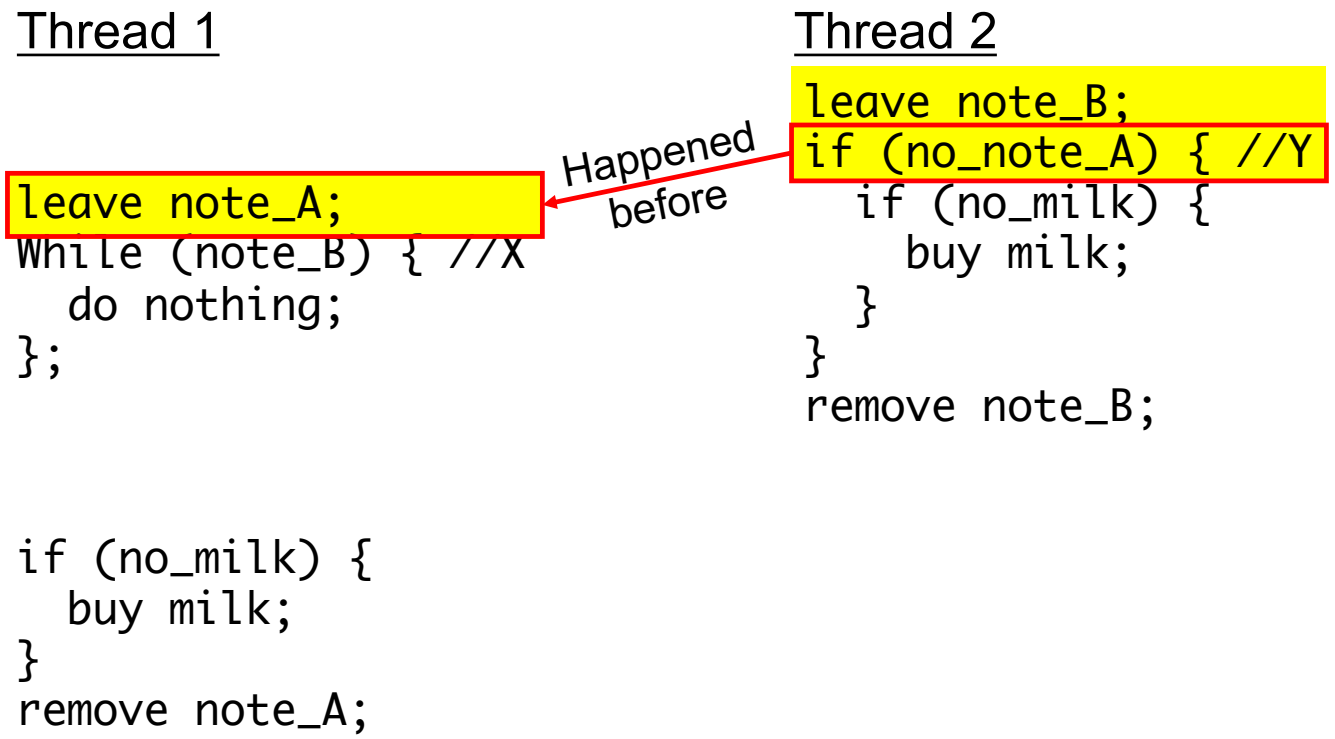
# Case 1

- "`leave note_A`" happens before "`if (no_note_A)`"



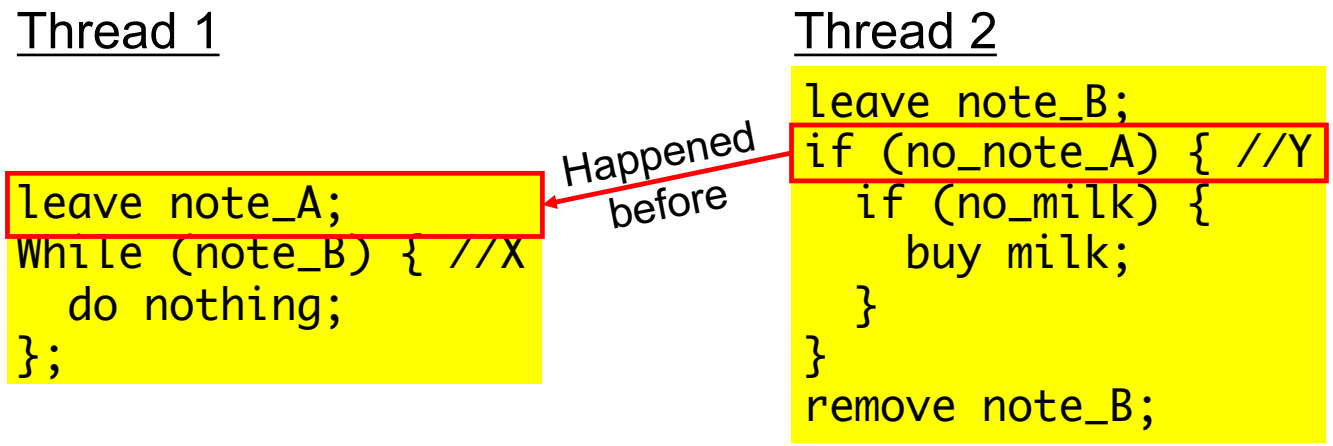
# Case 2

- "`if (no_note_A)`" happens before "`leave note_A`"



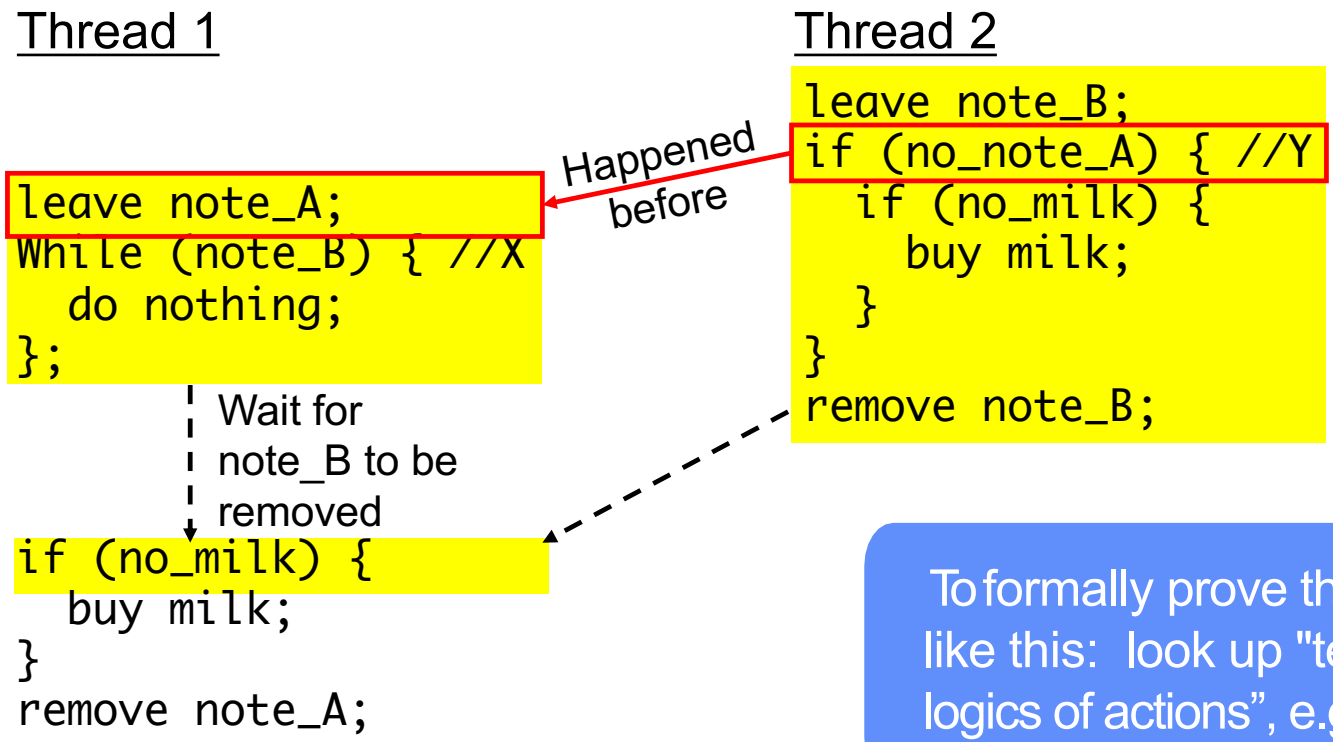
# Case 2

- "`if (no_note_A)`" happens before "`leave note_A`"



# Case 2

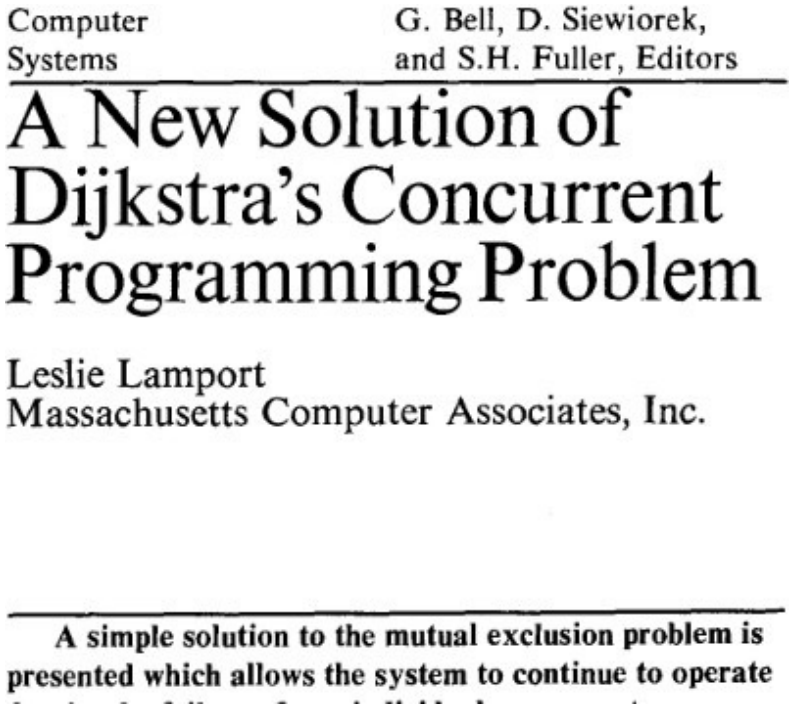
- "`if (no_note_A)`" happens before "`leave note_A`"




To formally prove things like this: look up "temporal logics of actions", e.g. TLA+

# This generalizes to n threads…

- Leslie Lamport's "Bakery Algorithm" (1974)



Computer Systems

G. Bell, D. Siewiorek, and S.H. Fuller, Editors

A New Solution of Dijkstra's Concurrent Programming Problem

Leslie Lamport
Massachusetts Computer Associates, Inc.

**A simple solution to the mutual exclusion problem is presented which allows the system to continue to operate**

# This generalizes to n threads…

- Solution #3 works, but it's really unsatisfactory
  - Really complex – even for this simple an example
    - Hard to convince yourself that this really works
  - A's code is different from B's – what if lots of threads?
    - Code would have to be slightly different for each thread
  - While A is waiting, it is consuming CPU time
    - This is called "busy-waiting"

# Too much milk: solution #4?

- Recall our target lock interface:
  - `acquire(&milklock)` – wait until lock is free, then grab
  - `release(&milklock)` – Unlock, waking up anyone waiting
  - These must be atomic operations – if two threads are waiting for the lock and both see it's free, only one succeeds to grab the lock

- Then, our milk problem is easy:

```
acquire(&milk_lock);
if (no_milk)
  buy milk;
release(&milk_lock);
```

# Where are we going with synchronization?

| | | |
|---|---|---|
| **Programs** | Shared Programs | |
| **Higher-level API** | Locks<br>Monitors | Semaphores<br>Send/Receive |
| **Hardware** | Load/Store<br>Test&Set | Disable Ints<br>Compare&Swap |

- Implement various higher-level synchronization primitives using atomic operations

# Reading for next lecture

- Operating System Concepts:
  - Chapter 6.4-6.6 (6.1-6.3 if you want to review today's content)

Or

- Operating System: Three Easy Pieces:
  - Chapters 28-31 (25-27 if you want to review today's content)

Thanks